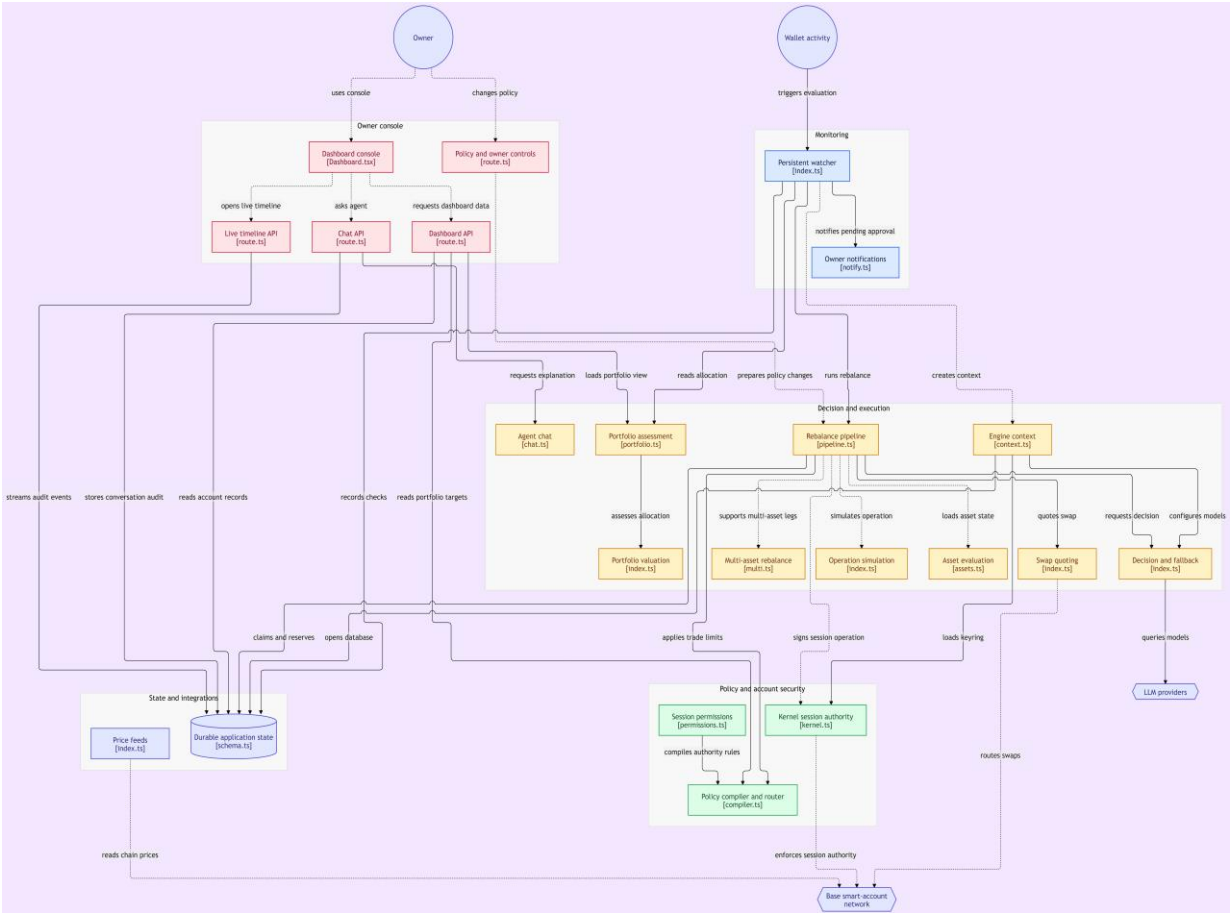
Guardian Agent Technical Documentation

Guardian is a policy-bound autonomous wallet. An LLM agent proposes rebalancing actions for a WETH/USDC portfolio. Deterministic code plans, prices, risk-checks and routes each proposal. A ZeroDev Kernel v3.3 smart account (ERC-4337, EntryPoint v0.7) independently enforces the authority delegated to the agent. Version 2 rebalances in both directions (sell WETH via `exactInputSingle`, buy WETH via `exactOutputSingle`) and handles multi-asset portfolios through a USDC hub, with one isolated session key for each tradable asset.

Partial-Step Autonomy: When a drift needs a $22 rebalance but the autonomous cap is $10, the agent doesn't fail or page the owner. Instead it executes one capped $10 step at Tier 1 (`PARTIAL_STEP`), then re-evaluates once the cooldown ends. Moves above 3× the cap go to the owner. Evidence: Execution Evidence, row 18.

3-Strike Fail-Safe AI Fallback (E8). If no model returns a valid decision, the owner's own target becomes a deterministic rule (`deterministic-fallback`, routed `RULE_BASED_FALLBACK`) for small, low-risk Tier 1 trades, so the portfolio isn't left unguarded. The 3rd consecutive failure fails closed (`AI_UNAVAILABLE_OR_INVALID`). Evidence: rows 15–17.

1. Architecture



Group	Component	Location	Responsibility
Owner console	Dashboard console [Dashboard.tsx]	apps/web/components/Dashboard.tsx	Owner-facing console; opens the live timeline, asks the agent, and requests dashboard data
Owner console	Policy and owner controls [route.ts]	apps/web/app/api/policy/route.ts	Accepts the owner's policy changes - band, caps, daily budget, slippage, session length
Owner console	Live timeline API [route.ts]	apps/web/app/api/stream/route.ts	Serves the live SSE event timeline the console streams from
Owner console	Chat API [route.ts]	apps/web/app/api/chat/route.ts	Routes the console's "ask agent" requests to Agent chat
Owner console	Dashboard API [route.ts]	apps/web/app/api/dashboard/route.ts	Serves portfolio, allocation and policy data to the console
Monitoring	Persistent watcher [index.ts]	apps/worker/src/index.ts	Watches wallet activity; on a meaningful change it runs the rebalance pipeline and creates engine context
Monitoring	Owner notifications [notify.ts]	apps/worker/src/notify.ts	Notifies the owner when a trade needs approval (Tier 2)
Decision and execution	Agent chat [chat.ts]	packages/agent/src/chat.ts	Answers the console's "explain this decision" requests in plain language
Decision and execution	Portfolio assessment [portfolio.ts]	packages/engine/src/portfolio.ts	Assesses the current allocation against the owner's band
Decision and execution	Portfolio valuation [index.ts]	packages/portfolio/src/index.ts	Integer, 18-decimal USD valuation that Portfolio assessment reads
Decision and execution	Rebalance pipeline [pipeline.ts]	packages/engine/src/pipeline.ts	Runs the trigger → plan → simulate → route → submit pipeline; reconciles unknown submissions
Decision and execution	Multi-asset rebalance [multi.ts]	packages/engine/src/multi.ts	Computes one rebalance leg per asset against the USDC hub
Decision and execution	Operation simulation [index.ts]	packages/simulation/src/index.ts	Runs EntryPoint.handleOps inside eth_simulateV1 before submission
Decision and	Asset evaluation [assets.ts]	packages/engine/src/assets.ts	Four-step on-chain check (metadata, Chainlink feed, USDC

Group	Component	Location	Responsibility
execution			pool, round-trip quote) that gates an asset live vs. paper
Decision and execution	Engine context [context.ts]	packages/engine/src/context.ts	Assembles the pipeline's context — prices, quote, risk, decision, keyring — and routes the swap
Decision and execution	Swap quoting [index.ts]	packages/dex/src/index.ts	QuoterV2 quote for the configured pool; sets the minimum output
Decision and execution	Decision and fallback [index.ts]	packages/agent/src/index.ts	Strict Zod decision contract; queries the configured LLM providers and runs the 3-strike rule-based fallback on failure
Policy and account security	Session permissions [permissions.ts]	packages/account/src/permissions.ts	Applies the compiled on-chain trade limits — call policy, rate limit, timestamp, gas
Policy and account security	Kernel session authority [kernel.ts]	packages/account/src/kernel.ts	Loads the session keyring, signs session operations, enforces authority on the Kernel account
Policy and account security	Policy compiler and router [compiler.ts]	packages/policy/src/compiler.ts	Compiles the owner's USD policy into on-chain restrictions and routes each trade to its authority tier
State and integrations	Price feeds [index.ts]	packages/oracle/src/index.ts	Reads Chainlink prices from the Base smart-account network
State and integrations	Durable application state [schema.ts]	packages/db/src/schema.ts	PostgreSQL schema: audit trail, account records, portfolio targets, budget claims and reservations
External	LLM providers	—	Groq, Gemini and OpenAI, queried by Decision and fallback
External	Base smart-account network	—	ERC-4337 EntryPoint, bundler and Kernel account reached by Kernel session authority and swap routing

Pinned stack: viem 2.57.5, @zerodev/sdk 5.5.10, @zerodev/permissions 5.6.3, Kernel v3.3, EntryPoint v0.7, call policy v0.0.5, Pimlico bundler and verifying paymaster, Next.js 16.4 with custom CSS modules, PostgreSQL, Drizzle 0.45.

2. Policy Compilation: Translating Human Financial Intent into Cryptographic Authority

The owner states intent in dollars and percentages:

```
{ "ethMinPct": 45, "ethMaxPct": 55, "targetEthPct": 53, "maxAutoTxUsd": 10,
  "dailyBudgetUsd": 25, "maxSlippageBps": 50, "sessionDurationHours": 24 }
```

The EVM can't read dollars, so the policy is compiled into restrictions the smart account can check. Each step is saved in `policy_compilations` for audit.

Step 1: Authorization price. Read Chainlink ETH/USD (`P_auth`, 8 decimals) and reject the price if it's stale.

Step 2: Buffered conversion. With autonomous cap `C` and appreciation buffer `b = 15%`, everything is computed in integers:

```
maxAmountInWei = floor( C_cents · 10^(18+dec) · 10000  /  (100 · P_auth · (10000 + b_bps)) )
```

Example: `C = $10` and `P_auth = $2,429.68` give `maxAmountInWei = 3,578,928,984,027,955` (about 0.00358 WETH, worth $8.70 at authorization).

Step 3: Bounded allowance. `allowance = maxAmountInWei × maxOps` (maxOps = 2). It's never unlimited.

Step 4: Session permission. The ceiling turns into four on-chain policies. Their order fixes the `PolicyFailed(i)` index:

#	Policy	Restriction
0	Call policy v0.0.5	Only `SwapRouter02.exactInputSingle`, with `value = 0`, `callType = CALL`. Field rules on the static tuple (offset `i × 32`): `tokenIn = WETH`, `tokenOut = USDC`, `fee = pool tier`, `recipient = smart account`, `amountIn ≤ maxAmountInWei`, `sqrtPriceLimitX96 = 0`
1	Timestamp	`validAfter`, `validUntil = authorization + 24 h`
2	Rate limit	`count = 2`, `interval = 0` (session-wide, never resets; proven)
3	Gas	Cumulative gas allowance 0.01 ETH; paymaster required and must be the approved Pimlico paymaster

v2 (two-way, as deployed). The table above shows the v1 permission, which produced the first evidence set. Permission v2 adds a second call-policy entry, `exactOutputSingle` with `tokenIn = USDC`, `tokenOut = WETH`, the same fee/recipient/price-limit pins and `amountInMaximum ≤ maxUsdcInRaw` (the USDC cap, 2% buffer). The rate limit becomes `RateLimitPolicyWithReset`, 6 ops per resetting 24 h window over a 7-day session. Allowances are capped per token at cap × 42. Multi-asset accounts get one such permission (one session key) per asset, since a single call policy can't hold two same-selector entries (`duplicate permissionHash`). You can see the live values for the current account in the console under **Guardrails → Inspect compiled on-chain permission spec**.

Step 5: Installation by the master. One master-signed batch performs `WETH.approve(router, allowance)`, `Kernel.installValidations(permission)` and `Kernel.grantAccess(permission, execute, true)`. Kernel v3.3 was chosen because v3.1 has no `grantAccess`, so a permission the master installed couldn't call `execute`.

What stays off-chain, by design. The token ceiling isn't a permanent USD ceiling: if WETH appreciates, the same amount is worth more dollars. That's why Guardian enforces the live USD cap and the daily budget off-chain, and sets `SESSION_REAUTH_REQUIRED` when the price rises 10% above `P_auth`. That sits below the 15% buffer, so the on-chain cap is still worth at most $10 when reauthorization kicks in. An exact on-chain USD cap would need an oracle-aware validator, which is out of scope for the MVP.

3. Core algorithms

Rebalance (integer, 18-decimal USD). WETH and USDC are valued using Chainlink prices. A rebalance triggers only when the portfolio is outside the band (`V_W · 100 > ethMaxPct · V_T` or `< ethMinPct · V_T`). Then, for target weight T:

```
overweight:  saleUsd = V_W − T · V_T     amountIn  = floor(saleUsd / P_WETH)       → exactInputSingle
underweight: buyUsd  = T · V_T − V_W     amountOut = floor(buyUsd / P_WETH)        → exactOutputSingle (amountInMaximum ≤ USDC cap)
```

Rounding goes down, so slightly less gets traded. USD amounts compared against caps round up (the conservative direction). Multi-asset portfolios compute one leg per asset against USDC and execute sells before buys.

Execution protection. `minOut = floor(quoteOut · (10000 − slippageBps) / 10000)`, where `quoteOut` comes from QuoterV2 for the exact pool and amount.

Risk. `R = 0.30·liquidity + 0.30·divergence + 0.20·size + 0.10·protocol + 0.10·token`, each scored 0–100. LOW ≤ 30, MEDIUM ≤ 60, HIGH > 60. Hard gates reject a trade whatever its score: divergence above the network threshold, USDC depeg above 2%, no liquidity, stale oracle, or L2 sequencer down.

Authority routing. Tier 3 is checked first (violations, hard gates, HIGH). Next, Tier 2: above the USD cap, above the daily budget, above the compiled token cap, session not active, MEDIUM risk, or AI `ESCALATE`. Otherwise it's Tier 1. The amount routed is the same in every tier; authority never changes the size of the rebalance.

Daily budget. Budgets follow the UTC calendar day. Reservations are made atomically under a row lock on the account, so concurrent reservations can't exceed the budget (tested with 4 parallel × $10 against $25: exactly 2 succeed). A reservation is released when the failure is definitive.

4. Lifecycle and recovery

```
TRIGGERED → PLANNING → SIMULATING → SUBMITTING → PENDING → CONFIRMED (cooldown 10 min)
                  ├→ PENDING_APPROVAL → REVALIDATING → MASTER_SIGNING → SUBMITTING → PENDING
                  ├→ REJECTED | NO_ACTION | WAIT | FAILED | EXECUTION_FAILED | CANCELLED
```

- **One active action per account.** A partial unique index enforces this, as do idempotent trigger keys, and both were tested under 8 concurrent claims.
- **Submission intent first.** The UserOperation is written to `user_operations` before it is sent. A bundler validation error is a definitive rejection. A timeout is `UNKNOWN`, and the account stays locked until it's reconciled from chain evidence.
- **Restart.** The worker reconciles pending operations before taking new work.
- **Fail closed.** If the LLM is unavailable or returns invalid output, the rule-based fallback may run a small LOW-risk Tier 1 trade; the 3rd consecutive failure pauses new autonomous actions. Stale oracles, failed quotes and failed simulations all stop the action.

5. Deployment and reproduction

You'll need Node 24 and pnpm (via corepack), plus credentials for Base RPC/WS, Pimlico, PostgreSQL and an LLM provider (Groq, Gemini or OpenAI), all listed in `.env.example`.

```
corepack pnpm install && corepack pnpm db:migrate && corepack pnpm test
node --env-file=.env --import tsx scripts/gate0/verify-addresses.ts      # on-chain address verification
node --env-file=.env --import tsx scripts/gate0/permission-harness.ts    # 20 permission cases (simulation)
node --env-file=.env --import tsx scripts/setup-account.ts               # deploy Kernel account (sponsored)
corepack pnpm guardian register && corepack pnpm guardian authorize
corepack pnpm worker                                                     # autonomous monitoring
corepack pnpm --filter @guardian/web build && corepack pnpm --filter @guardian/web start
```

Hosting targets: web on Vercel, the worker as a persistent process (e.g. Railway), and any managed PostgreSQL. On testnet, the sign-in card offers a no-wallet **Judge Sandbox** whose sessions can't change owner settings. Mainnet is refused unless `GUARDIAN_ALLOW_MAINNET=true`.